LABLEND: LAB EQUIPMENT BORROWING AND RETURN TRACKING SYSTEM

A PROJECT REPORT

Submitted by

SACHIN SARAVANAN

(240701449)

in partial fulfillment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**LABLEND: LAB EQUIPMENT BORROWING AND RETURN TRACKING SYSTEM**" is the bonafide work of **SACHIN SARAVANAN 240701449**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

DR.J.MANORANJINI
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

DR.J.JINU SOPHIA
COURSE FACULTY
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 11/11/2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **SACHIN SARAVANAN 240701449**, hereby declare that the project report titled "**LABLEND: LAB EQUIPMENT BORROWING AND RETURN TRACKING SYSTEM**", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, page, Deluge scripts and workflows — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report. No Zia or AI application builder feature was used to generate this application; every form, field, report and script was created manually in the Zoho Creator builder.

Place: Chennai

Date: 11/11/2026

SIGNATURE OF THE STUDENT

SACHIN SARAVANAN

240701449

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman MR.S.MEGANATHAN, Chairperson DR.M.THANGAM MEGANATHAN and Vice-Chairman MR.S.ABHAY SHANKAR MEGANATHAN for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, DR.S.N.MURUGESAN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank DR.J.MANORANJINI, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, DR.J.JINU SOPHIA, Associate Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank the laboratory staff of the department, whose day-to-day handling of lab equipment inspired this project, and my family and friends for their unwavering support.

SACHIN SARAVANAN

ABSTRACT

College laboratories lend oscilloscopes, multimeters, microcontroller kits, single-board computers and microscopes to students for lab sessions, mini projects and assignments. In most departments this lending is still recorded in a paper logbook or an informal spreadsheet: a student asks the lab staff in person, the staff member checks the shelf, writes the roll number and item in a register and agrees a return date verbally. Nobody can see at a glance which items are free, which are out, who has them and which are overdue, and missing items are usually discovered only during a periodic stock check. The problem is faced by students, lab staff and the faculty in charge of the laboratory.

LabLend was built on Zoho Creator because a single developer had to deliver a working, data-driven workflow application within one semester without writing a full web stack. The application has four forms (Equipment, Students, Requests and Returns) linked by lookup fields, five list reports including an Overdue Items report, and a Dashboard page that shows live counts of available, issued and overdue items. Deluge scripts fill request defaults on form load, block invalid dates and unavailable items on submission, and update the request and equipment status when an item is returned. Three Deluge functions run from conditional report buttons so that lab staff can Approve, Reject and Issue a request in one click, moving it through the lifecycle Pending → Approved / Rejected → Issued → Returned. Overdue status is derived by comparing the expected return date with the current date. No AI or Zia feature was used; every component was configured by hand.

The components were tested end to end with sample data: two requests were submitted, approved and issued, the dashboard moved from 5 / 0 / 0 to 3 / 2 / 1 (available / issued / overdue), one item was returned in good condition and the dashboard settled at 4 / 1 / 1 with the late request listed as overdue. The result is a single, traceable record for every loan, a controlled approval step, automatic status updates and an always-current view of lab inventory that replaces the paper register.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Laboratory Equipment; Inventory Tracking; Workflow Automation

PROJECT AT A GLANCE

Attribute	Details
Project Title	LabLend: Lab Equipment Borrowing and Return Tracking System
Application Domain	Education / College Laboratory Inventory and Asset Lending
Student Name & Register No.	SACHIN SARAVANAN – 240701449
Zoho Creator Edition	Zoho Creator (India data centre, zoho.in)
Application Name (Link Name)	LabLend (lablend)
Published / Live URL	https://creatorapp.zoho.in/sachinsaravanan.2024.cse1252/lablend/
Demo Credentials (per role)	Admin / Lab Staff: sachinsaravanan.2024.cse@rajalakshmi.edu.in Student: role simulated through the Requests form (no separate login)
Number of Forms / Subforms	4 / 0
Number of Reports (by type)	List: 5 | Kanban: 0 | Calendar: 0 | Pivot: 0 (total 5)
Number of Pages / Dashboards	1 (Dashboard: Available, Issued and Overdue counts + Overdue Items report)
Number of Workflows	Form: 4 | Report action: 3 | Schedule: 1 (saved, disabled on free plan) | Approval: 0 | Blueprint: 0 | Batch: 0
Deluge Functions / Custom Actions	3 functions (approveRequest, rejectRequest, issueRequest) / 3 custom report actions (Approve, Reject, Issue)
Integrations	None (e-mail, SMS and SSO kept out of scope)
Roles & Profiles Defined	Application owner (Super Admin) acting as Lab Staff; two invited users for review. Student and staff roles are represented in the workflow; separate permission profiles are listed as future work
Environments Used	Development (Zoho Creator) – Stage and Production not used
Date of Build and Test	09 November 2026 **Version: 1.0**

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 5
2.5 Proposed System 6
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 8
3.4 Hardware and Software Requirements 8
3.5 Use Case Model 9
SYSTEM DESIGN 10
4.1 System Architecture 10
4.2 Data Model 11
4.3 Form Design 11
4.4 Process and Workflow Design 12
4.5 User Interface and Navigation Design 13
4.6 Access Control Design 13
IMPLEMENTATION 15
5.1 Application Setup 15
5.2 Forms and Data Capture 15
5.3 Reports 18
5.4 Pages and Dashboards 20
5.5 Deluge Scripting 21
5.6 Workflows and Automation 24
5.7 Integrations 26
5.8 User Management and Role-Based Access 27
5.9 AI and Analytics Features 27
5.10 Pro-Code Extension 28

5.11 Mobile and Portal Access 28
TESTING AND DEBUGGING 29
6.1 Testing Strategy 29
6.2 Test Cases 29
6.3 Debugging Deluge Scripts 30
6.4 Defect Log 30
6.5 User Acceptance Testing 30
DEPLOYMENT, SECURITY AND GOVERNANCE 32
7.1 Environments and Deployment Strategy 32
7.2 Performance Optimisation 32
7.3 Security Features 33
7.4 Governance and Administration 33
RESULTS AND DISCUSSION 34
8.1 Application Output 34
8.2 Achievement of Objectives 35
8.3 Comparison with the Existing System 35
8.4 Limitations of the Low-Code Approach 36
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 36
CONCLUSION AND FUTURE ENHANCEMENTS 37
9.1 Conclusion 37
9.2 Future Enhancements 37
9.3 Learning Outcomes and Career Relevance 37
REFERENCES 39
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 40
APPENDIX B – COURSE OUTCOME MAPPING 42
APPENDIX C – DELUGE SOURCE CODE 43
APPENDIX D – USER MANUAL 45
APPENDIX E – EVALUATION RUBRIC 46
APPENDIX F – VIVA-VOCE RECORD SHEET 48

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 5
Table 3.1: Stakeholders and User Roles 7
Table 3.2: Functional Requirements 7
Table 3.3: Non-Functional Requirements 8
Table 3.4: Hardware and Software Requirements 8
Table 4.1: Field Specification of the Four Forms 11
Table 4.2: Role–Permission Matrix 13
Table 5.1: Reports Implemented 18
Table 5.2: Workflows Implemented 24
Table 5.3: Integrations Considered 26
Table 6.1: Test Cases 29
Table 6.2: Defect Log 30
Table 6.3: User Acceptance Feedback 30
Table 7.1: Performance Optimisation Applied 32
Table 7.2: Security Controls Implemented 33
Table 8.1: Mapping of Objectives to Outcomes 35
Table 8.2: Existing System versus LabLend 36
Table A.1: Feature Coverage Checklist 40
Table B.1: Mapping of Course Outcomes to Report Chapters 42
Table E.1: Evaluation Rubric for the Individual Project 46
Table E.2: Performance Level Descriptors 46
Table F.1: Viva-Voce Questions and Responses 48
Table F.2: Marks Summary 48

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 4
Figure 3.1: Use Case Diagram of the Application 9
Figure 4.1: System Architecture of the Application 10
Figure 4.2: Entity–Relationship Diagram 11
Figure 4.3: Workflow / Process Flow for the Request Lifecycle 12
Figure 4.4: Equipment Status Transitions 13
Figure 5.1: Zoho Creator Admin Dashboard with the LabLend Application 15
Figure 5.2: Application Builder (Design) and Navigation Sections 15
Figure 5.3: Equipment Form in Live Mode 16
Figure 5.4: Students Form in Live Mode 16
Figure 5.5: Requests Form with Defaults Set on Load 17
Figure 5.6: Equipment Lookup Listing Only Available Items 17
Figure 5.7: Requests Form in Builder Mode (Auto Number Starting at 1001) 17
Figure 5.8: Returns Form in Live Mode 18
Figure 5.9: Request Lookup Listing Only Issued Requests 18
Figure 5.10: All Equipment Report 19
Figure 5.11: All Students Report 19
Figure 5.12: All Requests Report 19
Figure 5.13: Overdue Items Report 20
Figure 5.14: Custom Actions Configured on the All Requests Report 20
Figure 5.15: LabLend Dashboard Page (Live) 21
Figure 5.16: Dashboard in the Page Builder 21
Figure 5.17: Request Form Defaults Workflow in the Deluge Editor 22
Figure 5.18: Validate Request Dates Workflow 22
Figure 5.19: Process Return Workflow 23
Figure 5.20: issueRequest Function in the Deluge Editor 24
Figure 5.21: Form Workflows of the Requests Form 25
Figure 5.22: Report Workflows (Approve, Reject, Issue) 25
Figure 5.23: Approve Request Report Workflow Running a Deluge Function 26
Figure 5.24: Daily Overdue Check Schedule (Disabled on the Free Plan) 26
Figure 5.25: Users of the Zoho Creator Account 27
Figure 5.26: Phone Preview of the Application 28
Figure 7.1: Environments (Stage and Production) in the Admin Console 32
Figure 8.1: Request 1001 after Return (Detail View) 34
Figure 8.2: Returns Report Showing the Recorded Return 35

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSE	Computer Science and Engineering
DC	Data Centre
ECE	Electronics and Communication Engineering
ER	Entity–Relationship
FR	Functional Requirement
ID	Identifier
IoT	Internet of Things
KPI	Key Performance Indicator
NFR	Non-Functional Requirement
OAuth	Open Authorization
QR	Quick Response
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SSO	Single Sign-On
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Engineering departments run laboratories that own a large number of portable instruments and kits: digital multimeters, digital storage oscilloscopes, Arduino and Raspberry Pi boards, sensor kits, signal generators and digital microscopes. These items are used inside scheduled lab sessions, but students also borrow them for mini projects, hackathons, IoT assignments and final-year work that continue for days or weeks. Each loan is a small transaction with a borrower, an item, a purpose, a start date and an expected return date, and the laboratory has to know at every moment which items are on the shelf, which are with students and which are late [1].

LabLend is built for the laboratory of a college department. Two groups of people work with it: students, who browse what is available, raise a borrow request and follow its status, and lab staff (the lab assistant or faculty in charge, acting as administrator), who approve or reject requests, hand items over, record returns and watch for overdue items. Every request refers to one student and one piece of equipment, and every return refers to one issued request, so the data naturally forms a small relational model of equipment, students, requests and returns. The same pattern — a master list, a request, an approval, a hand-over and a return — appears in library circulation, tool cribs and IT asset desks, which makes it a good fit for a low-code workflow application [2], [3].

1.2 Problem Statement

In most departments, lab lending is recorded in a paper logbook kept at the lab desk, sometimes copied into a spreadsheet at the end of the week. Because the logbook is the only record, staff cannot quickly answer basic questions such as whether an oscilloscope is free, who currently holds the Raspberry Pi kit, or which loans passed their return date. Students cannot check availability without walking to the lab, and the faculty in charge sees problems only during the periodic stock verification.

- **No real-time availability:** the status of an item is known only by looking at the shelf or reading through the logbook, so students make wasted trips and staff answer the same question many times.

- **Informal approval:** requests are made verbally; there is no record of who approved a loan, for what purpose or for how long.
- **Unclear return dates:** the expected return date is agreed in conversation and rarely written down, so nobody can say objectively when a loan became late.
- **No overdue monitoring:** late items are found only during stock checks, often weeks after the due date, when the borrower may have finished the course.
- **No condition history:** damage at return is not recorded, so a faulty item may be lent again or the damage cannot be traced to a particular loan.

1.3 Objectives

- To design a normalised data model and four linked forms for equipment, students, borrow requests and returns.
- To implement the request lifecycle Pending → Approved / Rejected → Issued → Returned with Deluge scripts, form workflows and one-click report actions for lab staff.
- To keep the equipment status (Available, Issued, Under Repair) synchronised automatically with requests and returns, so that an item can never be issued twice.
- To flag overdue loans automatically by comparing the expected return date with the current date, and to record the condition of every returned item.
- To provide list reports and a Dashboard page that show live counts of available, issued and overdue items, with the overdue list on the same screen.
- To keep the application simple enough for lab staff to run and maintain without programming knowledge.

1.4 Scope of the Project

In scope: a Zoho Creator web application with four forms (Equipment, Students, Requests, Returns), five list reports, one Dashboard page, four form workflows (on load, on validate and on success), three report custom actions backed by three Deluge functions, a scheduled overdue-check script (saved for when the plan allows schedules), seed data for five equipment items and three students, and access from the web and the Zoho Creator mobile app.

Out of scope: fines or payments for late returns; e-mail or SMS notifications; login with the college SSO (the application owner acts as lab staff and the student role is

represented through the request form); barcode or QR scanning of items; purchase, depreciation and maintenance scheduling of equipment; and integration with external inventory or ERP systems. These items were deliberately excluded to keep the project focused on a working borrow-and-return cycle.

1.5 Justification for a Low-Code Approach

Low-code platforms let a small team build and change business applications through visual configuration, extended by a scripting layer where needed [4], [5], [6]. Zoho Creator suits LabLend for several reasons [7]. The application is data-and-process centric, so forms, lookup relationships, drop-downs and list reports come from configuration rather than code. Form events (load, validate, successful submission) and report actions provide exactly the hooks needed for request defaults, validation and one-click status changes. A page builder turns record counts into a dashboard without a charting library. Hosting, backups, authentication and a mobile app are provided without separate infrastructure. Finally, the Deluge scripting language [8] keeps the custom logic short and readable — the entire business logic of LabLend is under eighty lines — so a lab assistant or the next student developer can maintain it after the semester.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator with screenshots of the live application. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements. The appendices contain the feature checklist, course outcome mapping, complete Deluge source code, a user manual and the evaluation sheets.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing process was studied by observing how equipment is lent in a typical department laboratory. A student who needs an item comes to the lab desk and asks the lab staff whether it is free. The staff member checks the shelf and the paper stock register, writes the roll number, item and date in the issue logbook and hands the item over; the return date is agreed verbally. When the student returns the item, the staff member strikes off the logbook entry and, at best, notes any damage in the margin. Late or missing items are found only during the periodic stock check. Figure 2.1 shows this flow.

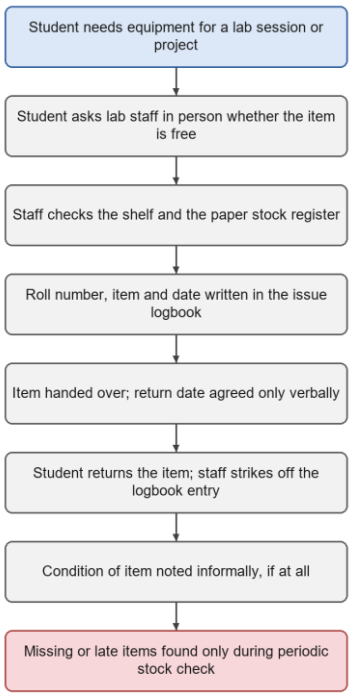


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- No single source of truth: availability, loans and returns live in a logbook, a stock register and people's memory.
- Every availability question needs a physical check of the shelf or a manual search of the logbook.

- Approvals are verbal, so there is no record of who allowed a loan, for what purpose or until when.
- The same item can be promised to two students because the register is not updated at the moment of hand-over.
- Overdue loans are discovered by chance or during stock verification, not on the day they become late.
- Return condition is not recorded in a structured way, so damaged items may be lent again.

2.3 Low-Code versus Traditional Development

Table 2.1 compares the two approaches for LabLend itself. The traditional effort figures are estimates for a comparable custom web application, not measured values.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Estimated 5–7 weeks to build the database, CRUD screens, approval logic, dashboard and hosting	About one week; forms, reports, page and workflows configured visually
Skills required	Front-end, back-end, database and DevOps skills	Data modelling, Creator builder and Deluge scripting
Mobile application	Separate mobile client or responsive design required	Available through the Zoho Creator mobile app
Security & access control	Authentication, sessions and permissions coded by hand	Built-in Zoho accounts, sharing and permission profiles
Maintenance & change	Code change, test and redeploy for each modification	Edit forms, workflows or Deluge in the builder; changes are live immediately

2.4 Literature Survey

Sources were chosen from three areas: inventory and asset-tracking practice in educational laboratories, low-code research and the Zoho Creator documentation. Table 2.2 summarises them.

Table 2.2: Summary of Literature Survey

S. No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	R. Nandhini et al., 2019	Laboratory inventory management systems in academic institutions	Manual registers cause stock mismatch and delays; digital tracking with status fields improves availability	Basis for the Equipment master and status values

S. No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
2	J. Kumar and P. Sharma, 2020	Web-based equipment lending and tracking for engineering labs	Request–approve–issue–return cycle with due dates reduces lost items	Shapes the request lifecycle in LabLend
3	A. Sahay et al., 2020	Supporting the understanding and comparison of low-code development platforms	Compares features of low-code platforms	Supports the platform choice and Table 2.1
4	A. C. Bock and U. Frank, 2021	Low-Code Platform	Defines low-code platforms and their concepts and limitations	Frames the low-code justification and limitations in Chapter 8
5	Zoho Corporation	Zoho Creator Help and Deluge reference	Documents forms, lookups, workflows, report actions, pages and schedules	Primary reference for every feature used

2.5 Proposed System

LabLend replaces the logbook with four linked forms in one Zoho Creator application. Each limitation in Section 2.2 is addressed directly: the Equipment form holds a status for every item, so availability is visible to everyone; the Requests form records the student, item, purpose and dates of each loan and starts in the Pending state; Approve, Reject and Issue buttons on the All Requests report give a recorded, one-click approval step; the equipment lookup on the request form lists only Available items and a validation script re-checks availability on submission, so an item cannot be promised twice; the Overdue Items report and the Dashboard compare the expected return date with today, so late loans are visible on the day they become late; and the Returns form captures the return date and condition and moves the item back to Available or to Under Repair.

The proposed system keeps the human decision points of the existing process — the lab staff still decides whether to lend an item and physically hands it over — but every decision now leaves a record and every status change is applied consistently by the platform.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator (application owner)	Developer and owner of the Zoho Creator app	Configure forms, reports, page, workflows and users; maintain master data
Lab Staff	Lab assistant or faculty in charge of the laboratory	Approve or reject requests, issue equipment, record returns and condition, watch overdue items
Student	Borrower from any department	Browse available equipment, submit a borrow request, follow request and return status
Head of Department / Reviewer	Faculty who reviews lab usage	View reports and the dashboard to see utilisation and overdue items

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall maintain an equipment master list with Equipment ID, name, category and status (Available / Issued / Under Repair)	Lab Staff	High
FR-02	The system shall maintain a student list with roll number, name, department and e-mail	Lab Staff	High
FR-03	The system shall let a student submit a borrow request with student, equipment, purpose, required date and expected return date	Student	High
FR-04	The system shall set the request date to today and the status to Pending automatically and prevent the user from editing them	Student	High
FR-05	The system shall list only Available equipment in the request form	Student	High
FR-06	The system shall reject a request whose expected return date is before the required date, or whose equipment is no longer available	Student	High
FR-07	The system shall allow lab staff to approve or reject a Pending request with one action	Lab Staff	High
FR-08	The system shall allow lab staff to issue an Approved request and mark the equipment as Issued	Lab Staff	High
FR-09	The system shall record a return with return date, condition and note, mark the request Returned and set the equipment to Available or Under Repair	Lab Staff	High
FR-10	The system shall flag a request as overdue when the expected return date has passed while it is Issued, and record whether a return was late	Lab Staff	High
FR-11	The system shall show an Overdue Items report of issued requests past their expected return date	Lab Staff, Reviewer	Medium

FR ID	Requirement	Role(s)	Priority
FR-12	The system shall show a dashboard with counts of available, issued and overdue items	All	Medium
FR-13	The system shall show only the actions valid for the current status (Approve / Reject for Pending, Issue for Approved)	Lab Staff	Medium
FR-14	The system shall be usable from a browser and from the Zoho Creator mobile app	All	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Forms, reports and the dashboard shall load within a few seconds for the expected volume of one department laboratory (hundreds of items, thousands of requests)
NFR-02	Data integrity	An item shall never be issued to two requests at the same time; status changes shall be applied by scripts, not typed by hand
NFR-03	Usability	Each form shall fit on one screen with clear labels, drop-downs instead of free text, and a navigation section per form
NFR-04	Availability	The application shall be reachable over the internet from the web and the mobile app without local installation
NFR-05	Maintainability	Business logic shall be kept in short, named Deluge scripts and functions that a non-specialist can read
NFR-06	Security	Only users invited to the application shall be able to view or change its data

3.4 Hardware and Software Requirements

LabLend runs on Zoho Creator (India data centre) and is scripted in Deluge. Users need a current Chrome, Edge or Firefox browser, or the Zoho Creator mobile app on Android or iOS, on a device with an internet connection. Development was carried out on a laptop with an Intel Core processor, 8 GB RAM and Windows 11. No local server, database or software installation is required for either development or use.

Table 3.4: Hardware and Software Requirements

Component	Requirement
Client hardware	Any laptop/desktop with 4 GB RAM or more, or an Android / iOS smartphone
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox (current version)
Platform	Zoho Creator, India data centre (creator.zoho.in)
Scripting	Deluge (Data Enriched Language for the Universal Grid Environment)
Mobile	Zoho Creator app for Android / iOS (optional)
Network	Internet connection; no local server or database

3.5 Use Case Model

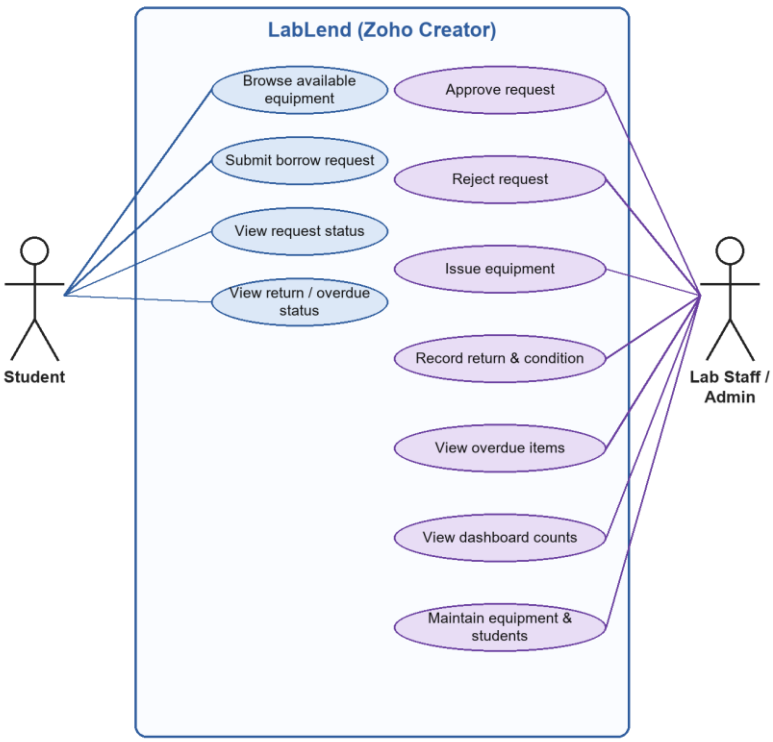


Figure 3.1: Use Case Diagram of the Application

Figure 3.1 shows the two actors and their main use cases. In the two key flows, the student submits a borrow request for an available item (FR-03 to FR-06), and the lab staff approves and issues it, later recording the return and its condition (FR-07 to FR-10). The dashboard and overdue use cases (FR-11, FR-12) read the data produced by these flows.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

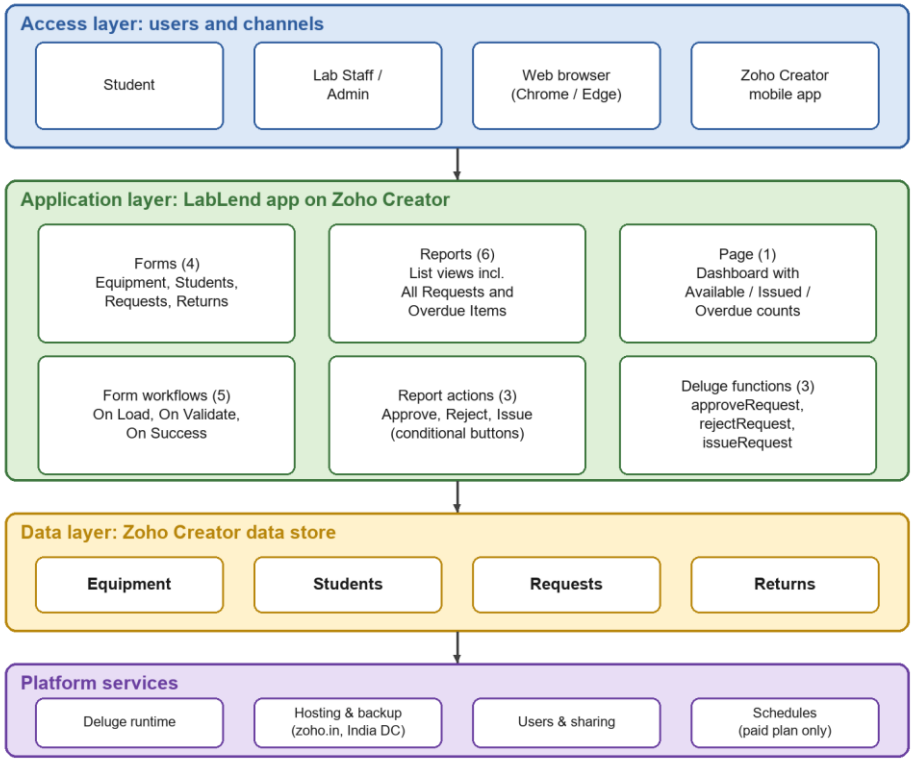


Figure 4.1: System Architecture of the Application

Figure 4.1 shows four layers. The access layer contains the student and the lab staff, who reach the application through a web browser or the Zoho Creator mobile app. The application layer is the LabLend Zoho Creator app: four forms, five reports, the Dashboard page, four form workflows, three report-action workflows and three Deluge functions. The data layer is the Zoho Creator data store that holds the records of the four forms. The platform services layer provides the Deluge runtime, hosting and backup in the India data centre, user management and the scheduler, which is available only on paid plans and is therefore saved but disabled.

4.2 Data Model

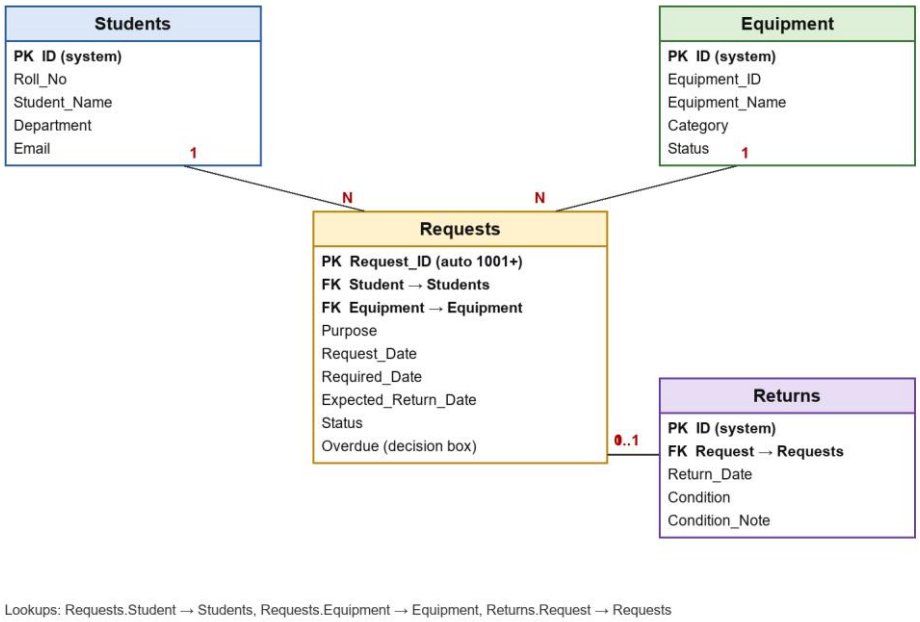


Figure 4.2: Entity–Relationship Diagram

Figure 4.2 shows the data model. Requests is the central form. It holds a Student lookup to Students and an Equipment lookup to Equipment, so one student can have many requests and one item can appear in many requests over time. Returns holds a Request lookup to Requests; each issued request has at most one return. The status fields on Equipment and Requests are the state of the process and are changed only by scripts after the initial entry. No subforms were needed because every repeating item (a request, a return) is a record in its own form so that it can have its own report, workflow and lookup.

4.3 Form Design

The field specifications of the four forms are given in Table 4.1. Requests uses an Auto Number identifier starting at 1001; the other forms use the system record ID and a human-readable key (Equipment ID, Roll No). Drop-downs are used for every controlled value so that reports and scripts can compare exact strings.

Table 4.1: Field Specification of the Four Forms

Form	Fields (Link Name – Field Type – Validation / Values)
Equipment	Equipment_ID – Single Line – mandatory (e.g. EQ-001) Equipment_Name – Single Line – mandatory Category – Drop Down – Electronics, Measurement, Computing, Optics – mandatory Status – Drop Down – Available, Issued, Under Repair – default Available – mandatory

Form	Fields (Link Name – Field Type – Validation / Values)
Students	Roll_No – Single Line – mandatory (e.g. 2024CSE101) Student_Name – Single Line – mandatory Department – Drop Down – CSE, ECE, EEE, IT, MECH Email – Email
Requests	Request_ID – Auto Number – starts at 1001 Student – Lookup (Students, display Roll No – Name) – mandatory Equipment – Lookup (Equipment, display Name, filter Status == Available) – mandatory Purpose – Multi Line – mandatory Request_Date – Date – set to today, read-only Required_Date – Date – mandatory Expected_Return_Date – Date – mandatory, ≥ Required_Date Status – Drop Down – Pending, Approved, Rejected, Issued, Returned – read-only Overdue – Decision Box – set by scripts, read-only
Returns	Request – Lookup (Requests, display Request ID, filter Status == Issued) – mandatory Return_Date – Date – mandatory, default today Condition_field – Drop Down – Good, Minor Damage, Needs Repair – default Good Condition_Note – Multi Line

4.4 Process and Workflow Design

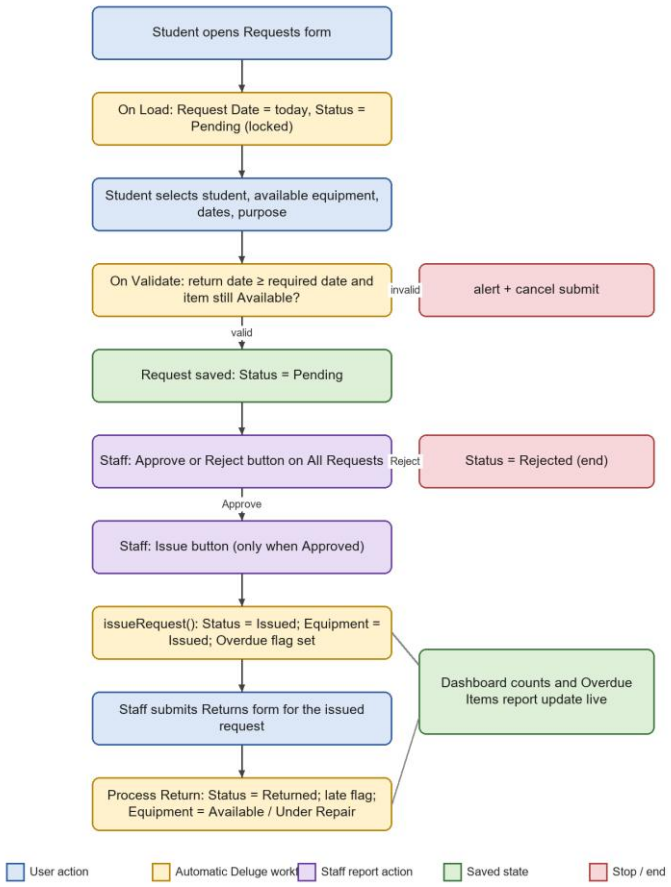


Figure 4.3: Workflow / Process Flow for the Request Lifecycle

Figure 4.3 shows the intended path of a request. Blue boxes are user actions on forms, yellow boxes are Deluge workflows that run automatically, purple boxes are report actions run by lab staff, green boxes are saved states and red boxes stop the flow. A request begins as Pending after passing validation, is approved or rejected by staff, is

issued (which also marks the equipment as Issued and sets the overdue flag if the return date has already passed), and ends when the Returns form is submitted.

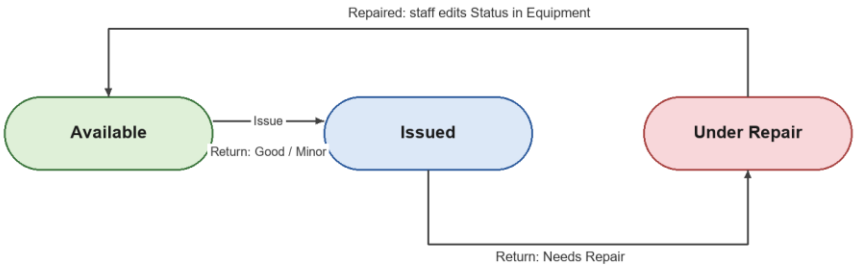


Figure 4.4: Equipment Status Transitions

Figure 4.4 shows the matching life of an equipment item. An item moves from Available to Issued only through the Issue action, and back to Available or to Under Repair only through a return. Moving an item from Under Repair to Available is a deliberate manual edit by lab staff after the repair.

4.5 User Interface and Navigation Design

The navigation is organised as one section per form in the left menu. Each section contains the form and its reports: Equipment (Equipment, All Equipment), Students (Students, All Students), Requests (Requests, All Requests, Overdue Items) and Returns (Returns, Returns Report), followed by the Dashboard page. Forms use a single-column layout with labels on the left. The Dashboard is designed as a single-screen overview: a panel with three large coloured counts (green for available, blue for issued, red for overdue) and, below it, the embedded Overdue Items report so that staff can act on late loans without changing screens.

4.6 Access Control Design

Access is designed around two logical roles. Table 4.2 shows the intended permissions on each component. A = Add, V = View, E = Edit, run = can run the Approve, Reject and Issue actions, — = no access. In the current build the application owner performs the lab staff role and the student role is exercised through the Requests form; configuring separate permission profiles for these roles is listed under future enhancements (Section 9.2).

Table 4.2: Role–Permission Matrix

Component	Administrator	Lab Staff	Student	Reviewer
Equipment form / All Equipment	A V E	A V E	V	V
Students form / All Students	A V E	A V E	—	V

Component	Administrator	Lab Staff	Student	Reviewer
Requests form	A V E	V E	A	V
All Requests (Approve / Reject / Issue)	V + run	V + run	V (own)	V
Overdue Items report	V	V	—	V
Returns form / Returns Report	A V E	A V	V (own)	V
Page: Dashboard	V	V	V	V
Workflows, functions, users	A V E	—	—	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application LabLend was created from scratch in Zoho Creator under the developer's account in the India data centre (creator.zoho.in), with the link name `lablend`, the date format dd-MMM-yyyy and the Asia/Kolkata time zone. The free plan allows one application, so an older practice application was deleted first to free the slot. No template, sample application or AI application builder was used. The four forms were created first, followed by the seed data, the reports, the form workflows, the Deluge functions and report actions, and finally the Dashboard page.

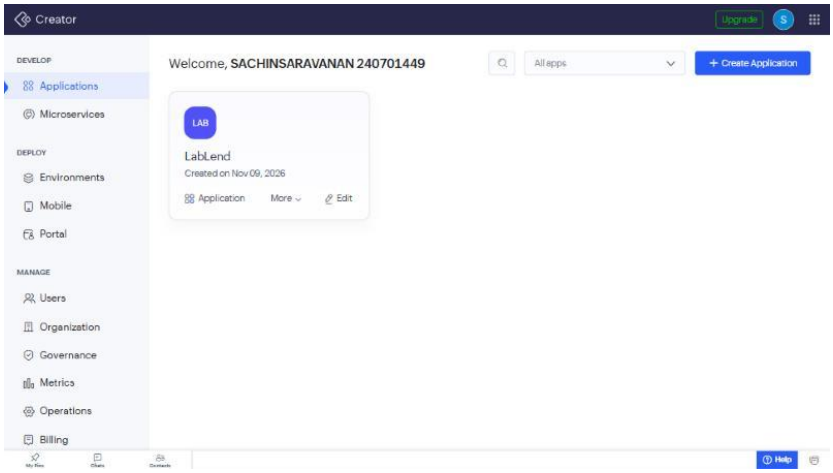


Figure 5.1: Zoho Creator Admin Dashboard with the LabLend Application

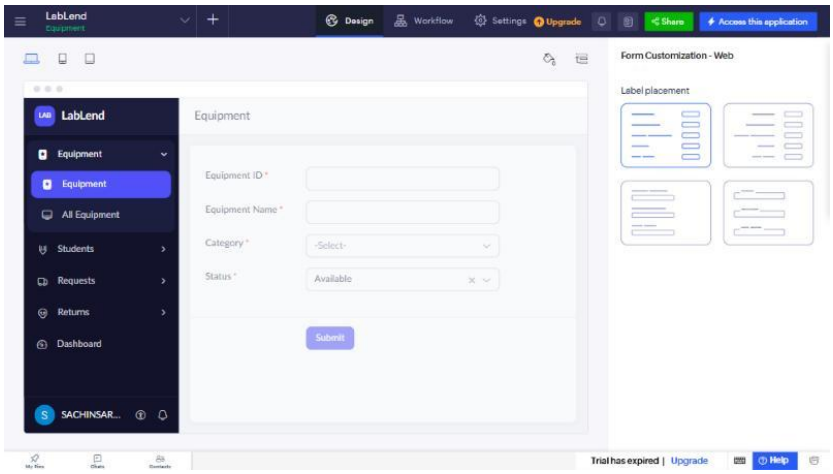


Figure 5.2: Application Builder (Design) and Navigation Sections

5.2 Forms and Data Capture

All four forms use mandatory flags on key fields, drop-down fields for controlled vocabularies (category, equipment status, department, request status and return

condition) and lookup fields for relationships. Together they use eight distinct field types: Single Line, Multi Line, Email, Drop Down, Date, Auto Number, Lookup and Decision Box. Equipment and Students are master forms filled by lab staff; Requests and Returns are transaction forms.

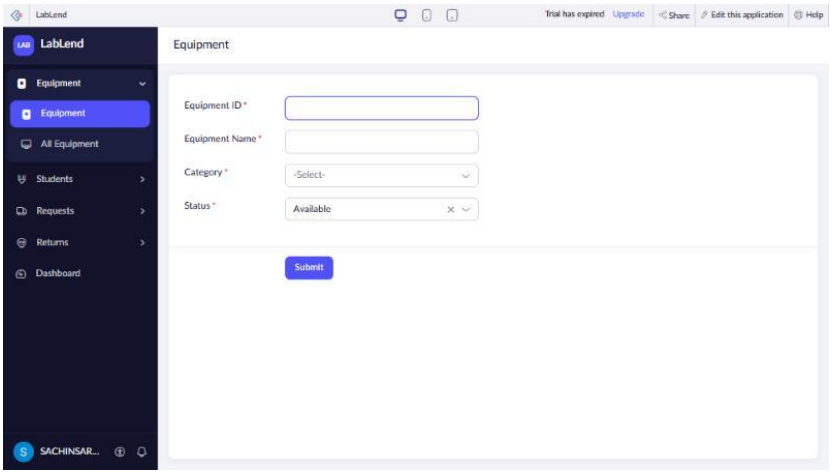


Figure 5.3: Equipment Form in Live Mode

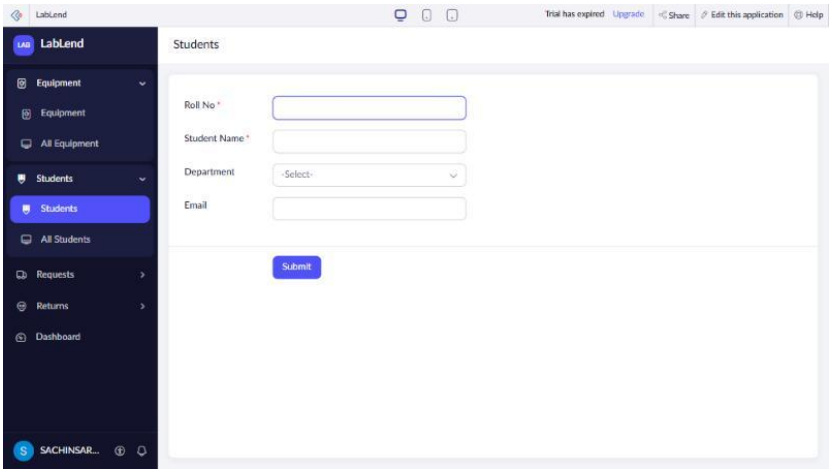


Figure 5.4: Students Form in Live Mode

The Requests form is where most of the behaviour lives. When it opens, the Request Form Defaults workflow sets the Request Date to today, the Status to Pending and the Overdue flag to false, and disables all three fields, so a student cannot change them. The Equipment lookup carries a filter `Status == "Available"`, so the drop-down shows only items currently on the shelf (in Figure 5.6 the Raspberry Pi 4, which is issued, is not offered). The Student lookup displays the roll number and name together so the right student is chosen.

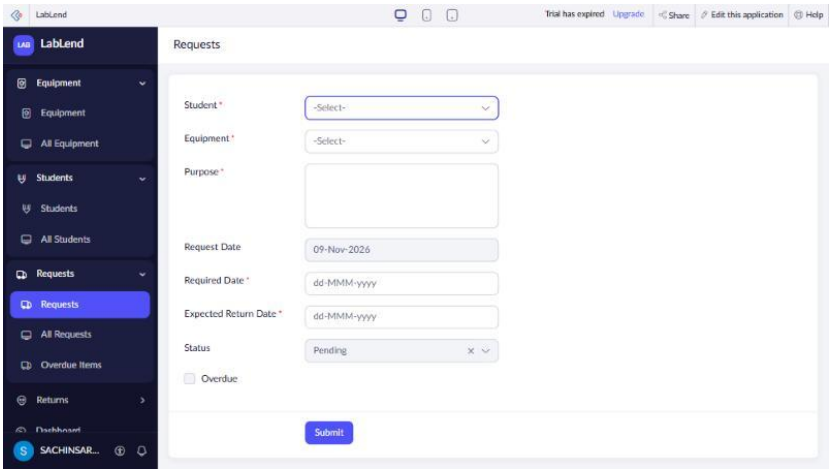


Figure 5.5: Requests Form with Defaults Set on Load

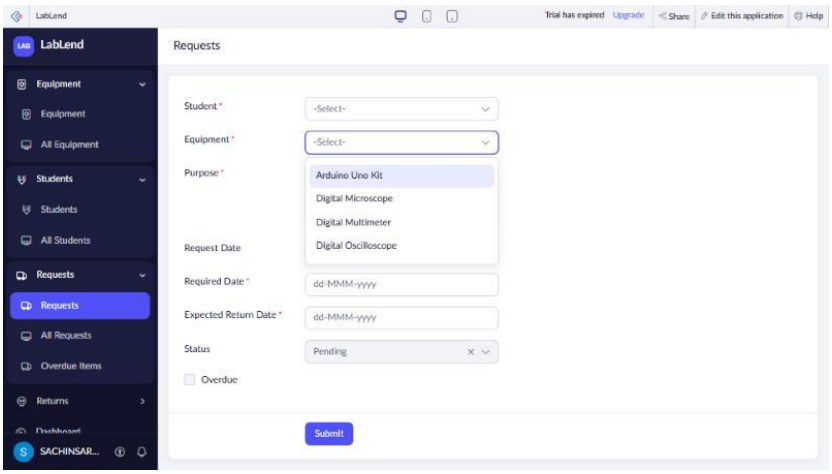


Figure 5.6: Equipment Lookup Listing Only Available Items

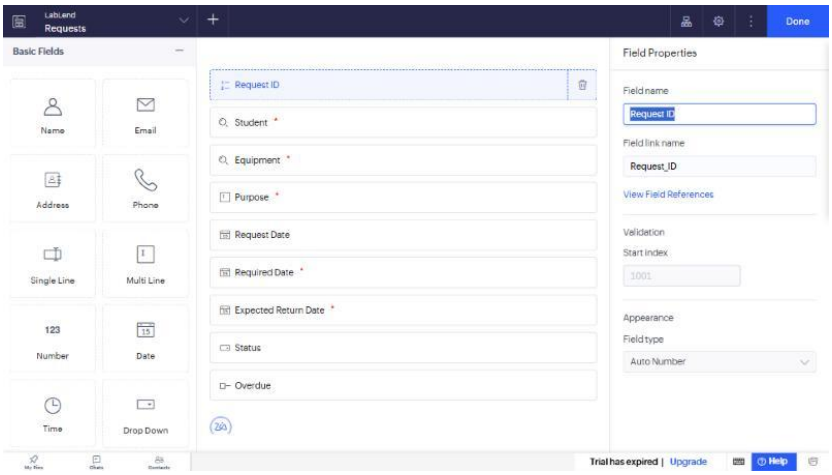


Figure 5.7: Requests Form in Builder Mode (Auto Number Starting at 1001)

The Returns form is used by lab staff. Its Request lookup is filtered to `Status == "Issued"`, so only loans that are actually out can be returned (Figure 5.9 shows only request 1002). The Return Form Defaults workflow sets Return Date to today, and Condition defaults to Good.

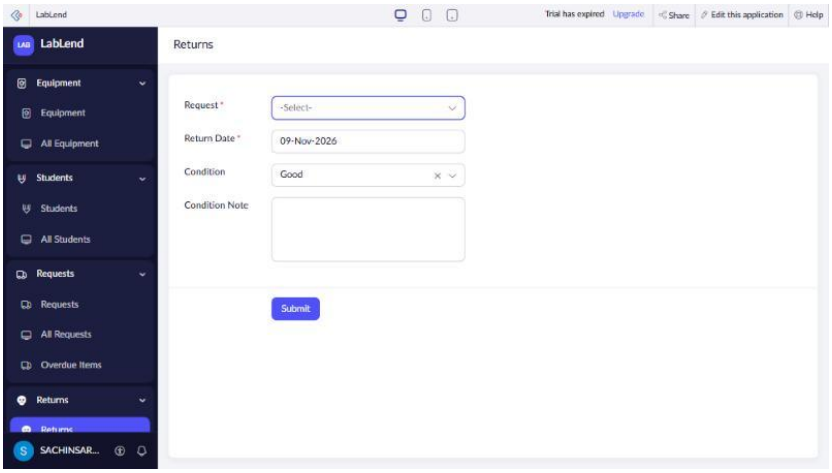


Figure 5.8: Returns Form in Live Mode

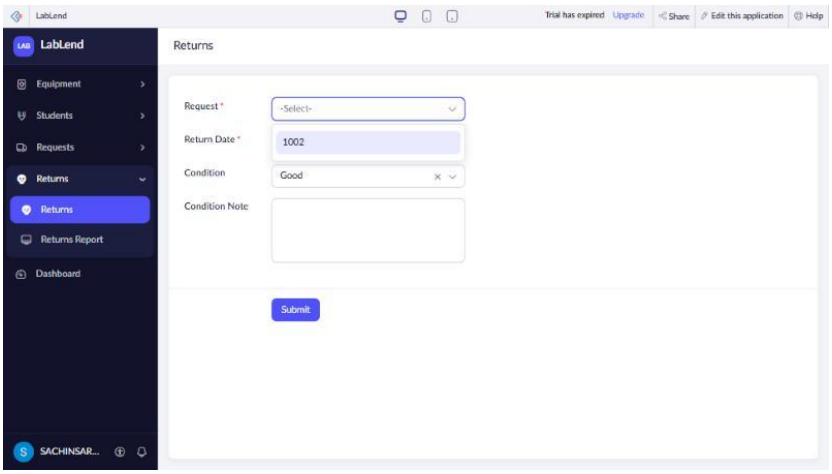


Figure 5.9: Request Lookup Listing Only Issued Requests

5.3 Reports

Five list reports were created (Table 5.1). Four are the default reports of the forms with the column order and sorting adjusted; the fifth, Overdue Items, is a separate list report on the Requests form with the filter `Status == "Issued" && Expected_Return_Date < zoho.currentdate`, so it always reflects today's date without any stored flag. The All Requests report carries three custom actions — Approve, Reject and Issue — that appear in the record menu next to Edit, Duplicate and Delete.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Display / Filter	Custom Actions
All Equipment	List	Equipment	All items with ID, name, category, status	—
All Students	List	Students	Roll No, name, department, e-mail	—
All Requests	List	Requests	All requests incl. dates, status and overdue flag	Approve; Reject (Pending only); Issue (Approved only)

Report Name	Type	Source Form	Display / Filter	Custom Actions
Overdue Items	List	Requests	Status == Issued && Expected_Return_Date < today	—
Returns Report	List	Returns	Request, return date, condition, note	—

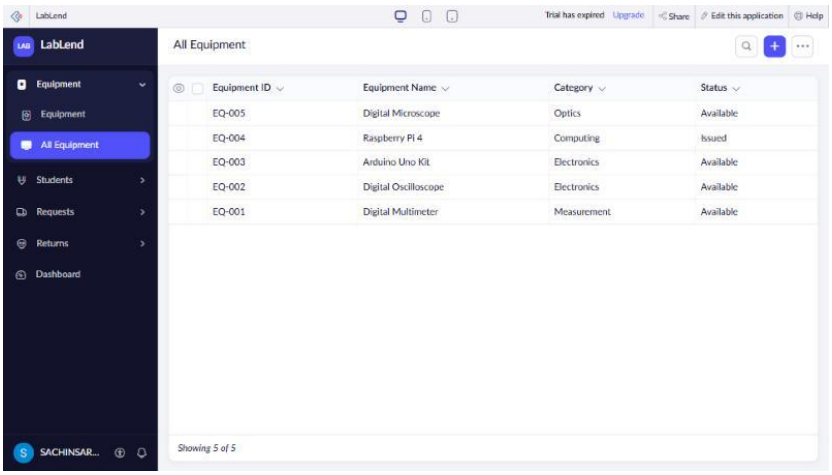


Figure 5.10: All Equipment Report

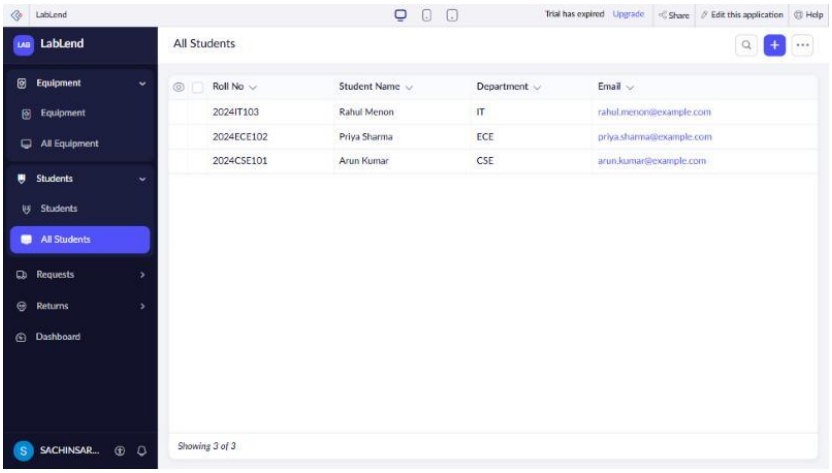


Figure 5.11: All Students Report

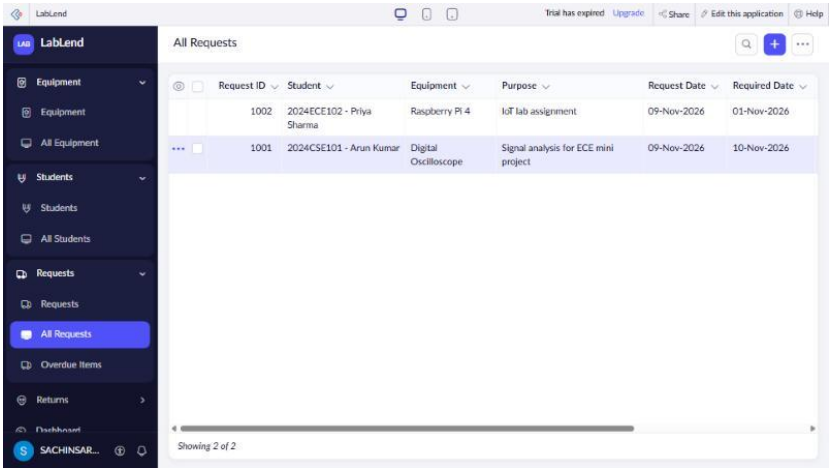


Figure 5.12: All Requests Report

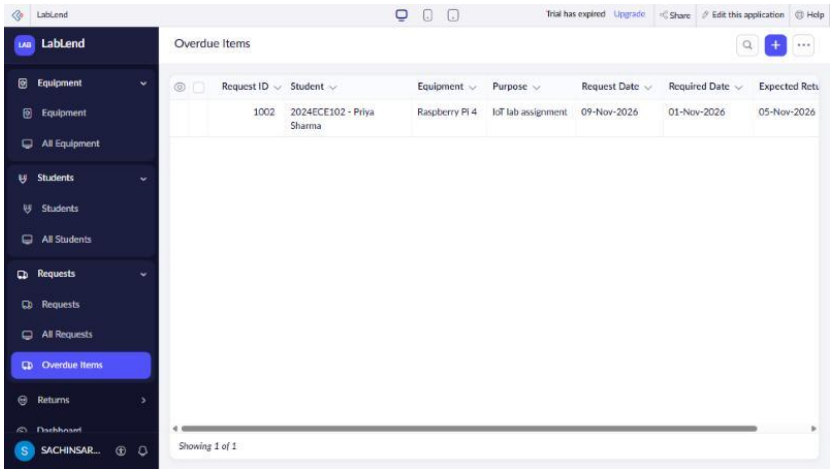


Figure 5.13: Overdue Items Report

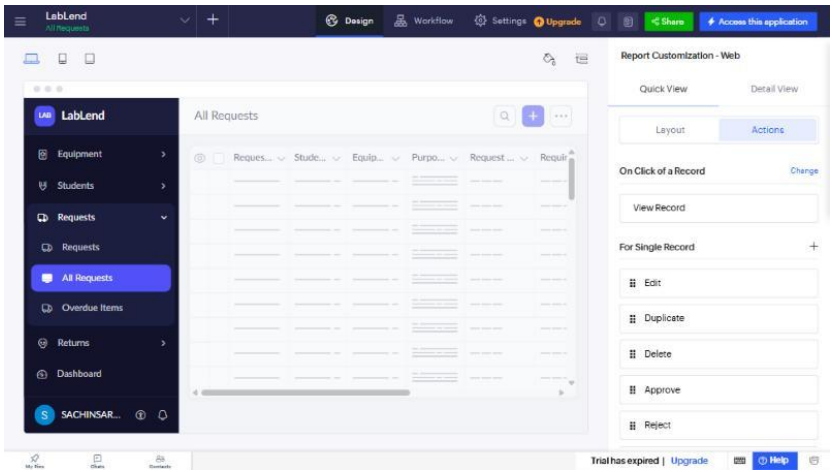


Figure 5.14: Custom Actions Configured on the All Requests Report

5.4 Pages and Dashboards

The Dashboard page was built in the page builder. It contains a panel element with three columns, each showing a label and a record count calculated by the platform with criteria: Available Items counts Equipment records with `Status == "Available"`, Issued Items counts Equipment with `Status == "Issued"`, and Overdue Items counts Requests with `Status == "Issued" && Expected_Return_Date < zoho.currentdate`. The counts use green, blue and red text so the state of the lab is readable from across the room. Below the panel, the Overdue Items report is embedded so that staff can open a late request directly from the dashboard. The page layout is stored in ZML (Zoho Markup Language), and the panel was written in ZML to set the criteria precisely.

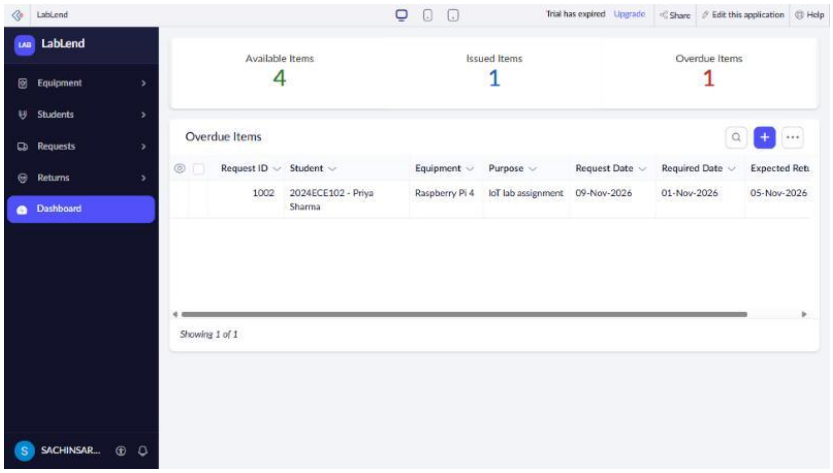


Figure 5.15: LabLend Dashboard Page (Live)

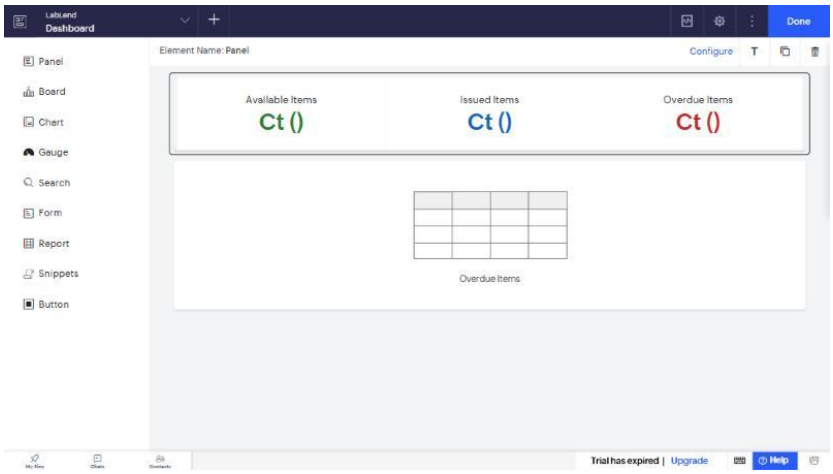


Figure 5.16: Dashboard in the Page Builder

Code 5.1: Dashboard panel in ZML (one of the three count columns)

```
<pc padding='20px' bgColor='#FFFFFF' width='33%' hAlign='center'>
  <pr><pc><text color='#333333' size='16px' type='Text'
           value='Overdue Items'> </text></pc></pr>
  <pr><pc><text color='#C62828' size='36px' bold='true' type='Form Data'
        criteria='Status == "Issued" && Expected_Return_Date &lt;
zoho.currentdate'
        value='thisapp.Requests.ID.count'> </text></pc></pr>
</pc>
```

5.5 Deluge Scripting

Deluge [8] is used in four form workflows, three functions and one schedule. Each script is short and has a single responsibility. The complete code is listed in Appendix C; the most important scripts are shown below.

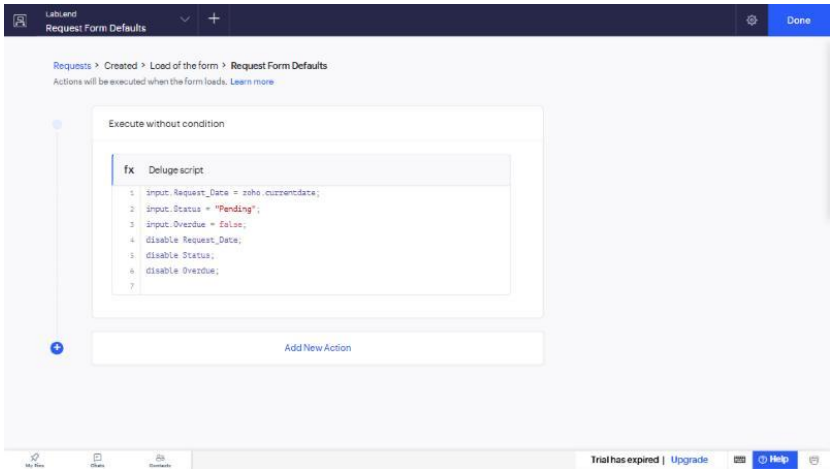


Figure 5.17: Request Form Defaults Workflow in the Deluge Editor

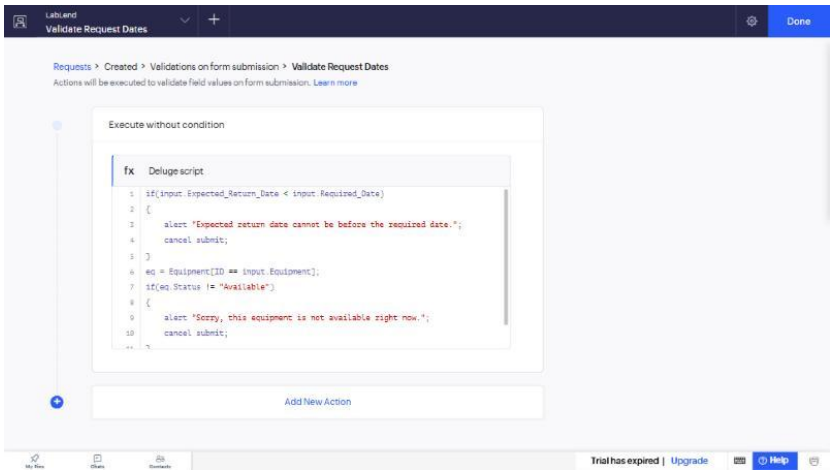


Figure 5.18: Validate Request Dates Workflow

Code 5.2: Process Return – On Successful Submission of the Returns form (DS-04)

```
req = Requests[ID == input.Request];
req.Status = "Returned";
req.Overdue = input.Return_Date > req.Expected_Return_Date;
eq = Equipment[ID == req.Equipment];
if(input.Condition_field == "Needs Repair")
{
    eq.Status = "Under Repair";
}
else
{
    eq.Status = "Available";
}
```

Code 5.3: issueRequest function (DS-07)

```
void issueRequest(int reqId)
{
    r = Requests[ID == reqId];
    if(r.Status == "Approved")
    {
        eq = Equipment[ID == r.Equipment];
        if(eq.Status == "Available")
        {
            r.Status = "Issued";
            r.Overdue = r.Expected_Return_Date < zoho.currentdate;
```

```
            eq.Status = "Issued";
        }
    }
}
```

How the scripts work. DS-01 runs when the request form loads and fixes the values a student must not choose. DS-02 runs when the form is submitted, before the record is saved: the first check compares the two dates, the second fetches the chosen Equipment record and refuses the request if another student took the item after the form was opened; `alert` shows the message and `cancel submit` stops the save. DS-04 runs after a return is saved: it fetches the linked request, marks it Returned, records whether it came back late by comparing dates, and then sets the equipment to Under Repair or Available depending on the condition. The three functions DS-05 to DS-07 receive the record ID from the report action, fetch the request and change its status only when the current status allows it, so pressing a button twice or on the wrong record has no effect. In Deluge, assigning to a field of a fetched record updates the record directly, so no explicit save call is needed.

Result observed. Opening the form showed today's date and Pending in locked fields; submitting a return date earlier than the required date showed the alert and saved nothing; issuing request 1002, whose expected return date was in the past, set its overdue flag to true; and returning request 1001 in Good condition set it to Returned and its oscilloscope back to Available.

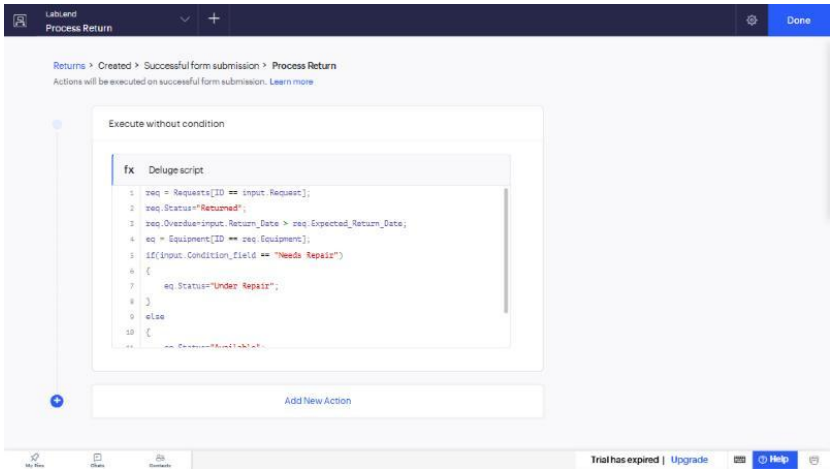


Figure 5.19: Process Return Workflow

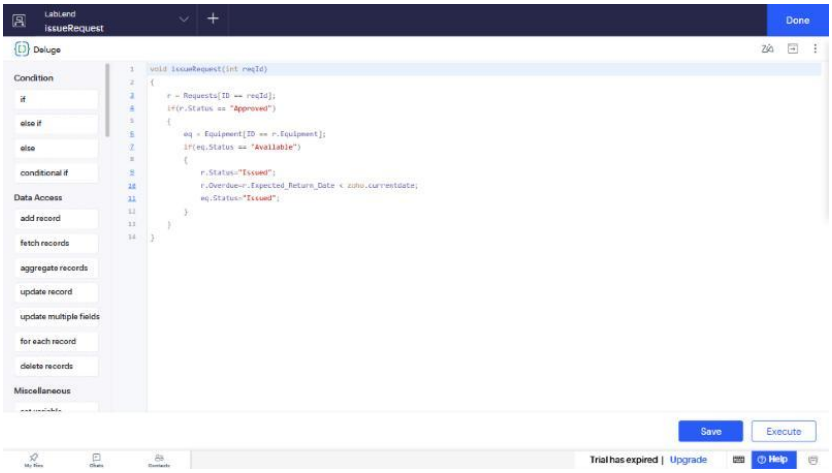


Figure 5.20: issueRequest Function in the Deluge Editor

5.6 Workflows and Automation

Table 5.2 lists every workflow. Form workflows prepare and validate the request and process the return; report workflows connect the three buttons on the All Requests report to the Deluge functions; and the schedule is prepared to refresh the stored overdue flag every day.

Table 5.2: Workflows Implemented

Workflow Name	Type	Trigger / Event	Action(s)	Script Ref.
Request Form Defaults	Form	Requests – Created → Load of the form	Set date, Pending, overdue = false; lock fields	DS-01
Validate Request Dates	Form	Requests – Created → Validations on form submission	Check dates and availability; alert and cancel	DS-02
Return Form Defaults	Form	Returns – Created → Load of the form	Return Date = today	DS-03
Process Return	Form	Returns – Created → Successful form submission	Request Returned; late flag; equipment status	DS-04
Approve Request	Report action	User trigger in All Requests; shown when Status == Pending	Run approveRequest(ID)	DS-05
Reject Request	Report action	User trigger in All Requests; shown when Status == Pending	Run rejectRequest(ID)	DS-06
Issue Equipment	Report action	User trigger in All Requests; shown when Status == Approved	Run issueRequest(ID)	DS-07
Daily Overdue Check	Schedule	Daily at 00:05 from 10-Nov-2026	Refresh Overdue flag of Issued requests (disabled)	DS-08

Conditional actions. Each report workflow has a display condition, so the Approve and Reject buttons appear only on Pending requests and the Issue button only on Approved requests. This enforces the lifecycle in the user interface, while the status checks inside the functions enforce it again on the server. **Approvals and Blueprint.** The single approval decision of LabLend is made through the Approve and Reject

actions, which was simpler for one approver than a formal approval process; a Blueprint was not needed because the lifecycle has only one path. **Schedule.** The Daily Overdue Check schedule was created and its script saved, but the free plan shows “Upgrade to Enable Workflow”, so it is disabled. The application does not depend on it: the Overdue Items report and the dashboard calculate overdue status live from the dates.

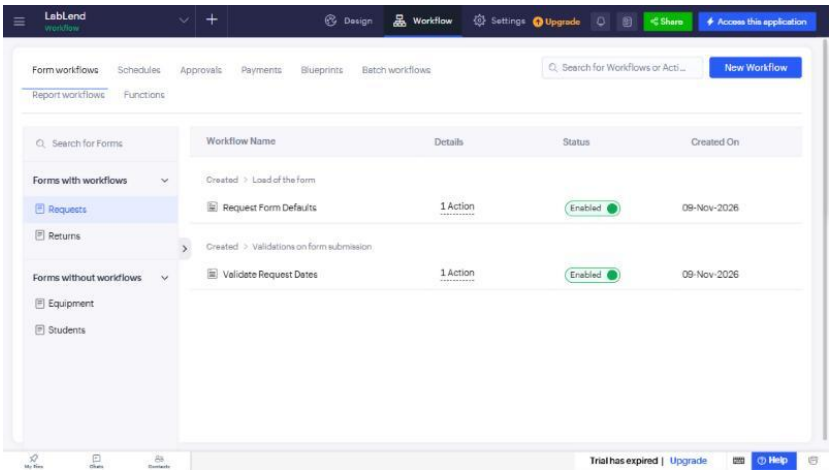


Figure 5.21: Form Workflows of the Requests Form

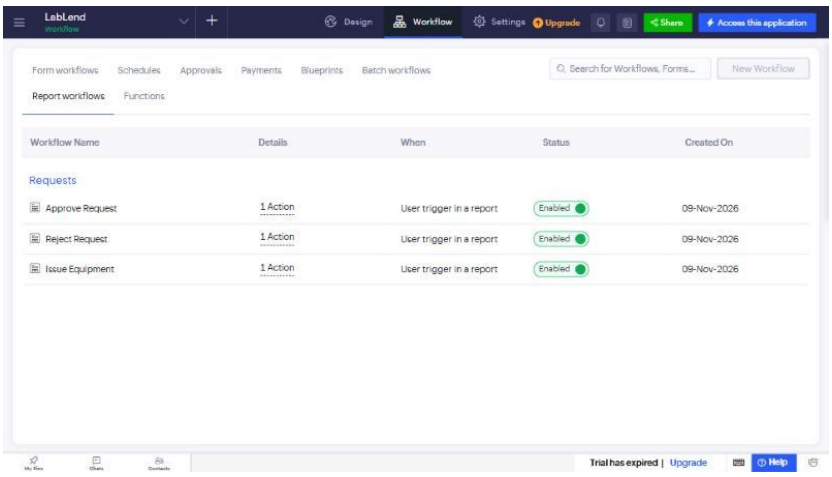


Figure 5.22: Report Workflows (Approve, Reject, Issue)

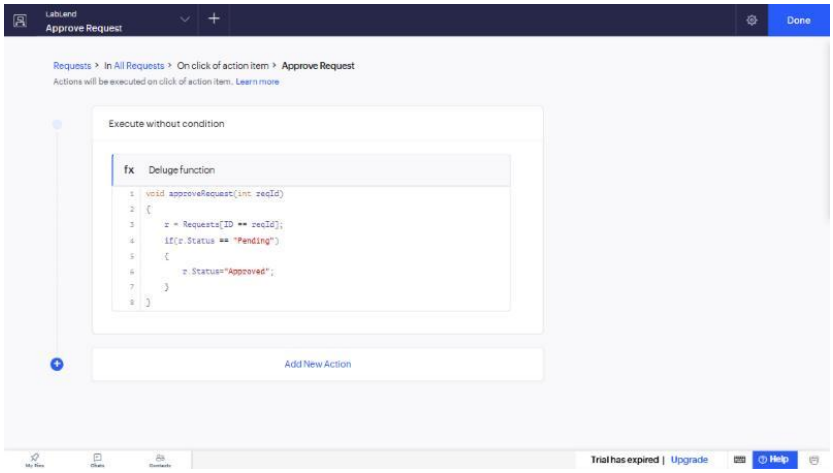


Figure 5.23: Approve Request Report Workflow Running a Deluge Function

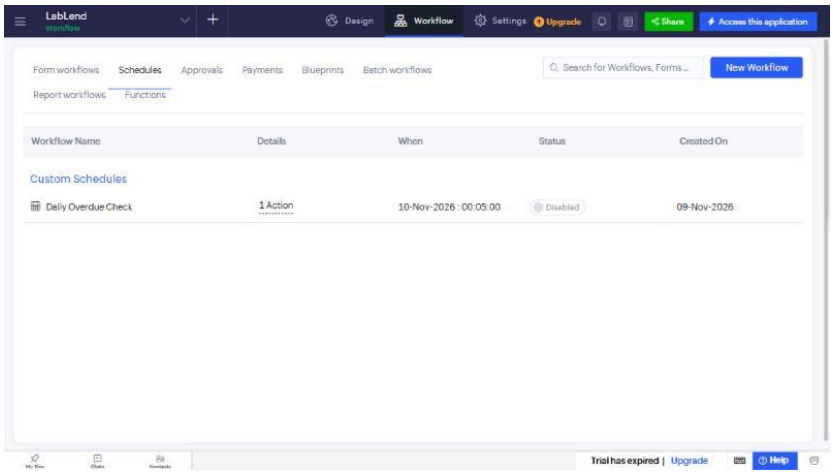


Figure 5.24: Daily Overdue Check Schedule (Disabled on the Free Plan)

5.7 Integrations

No external integration was implemented in LabLend. The project brief placed e-mail and SMS notifications and login with the college SSO out of scope, and the core value of the application — an always-current record of loans — does not depend on another system. No connection, `invokeurl` call, webhook or Creator REST API was used, so no Postman test was performed. The data model was nevertheless designed so that integrations can be added without restructuring: the Students form already stores an e-mail address that a future `sendmail` reminder can use (Section 9.2), and the Equipment ID is a stable key that could be shared with a college inventory or ERP system through the Creator REST API.

Table 5.3: Integrations Considered

Integration	Service / Endpoint	Status	Reason / Future Use
E-mail reminders	Zoho Creator sendmail task	Not implemented	Out of scope; Students.Email ready for use
College SSO login	SAML / OAuth identity provider	Not implemented	Out of scope; Zoho accounts used instead

Integration	Service / Endpoint	Status	Reason / Future Use
Inventory / ERP sync	Creator REST API or webhook	Not implemented	Equipment ID can act as shared key
Zoho Analytics	Native connector	Not implemented	Dashboard counts sufficient for current volume

5.8 User Management and Role-Based Access

Users are managed in the Users area of the Zoho Creator admin console. The developer account is the Super Admin and owns the application; it performs the lab staff role in the current build. Two further users were invited to the application for review and testing and appear with the status Inactive until they accept the invitation (Figure 5.25). Access to the application data is limited to these invited users, so the data is not public. The project brief asked for a simple role field instead of a real login for students; in the current build students and staff therefore see the same menu, and the separation of duties shown in Table 4.2 is enforced by the workflow (only status-appropriate buttons, read-only status fields) rather than by permission profiles. Creating Student and Lab Staff profiles is the first item in Section 9.2.

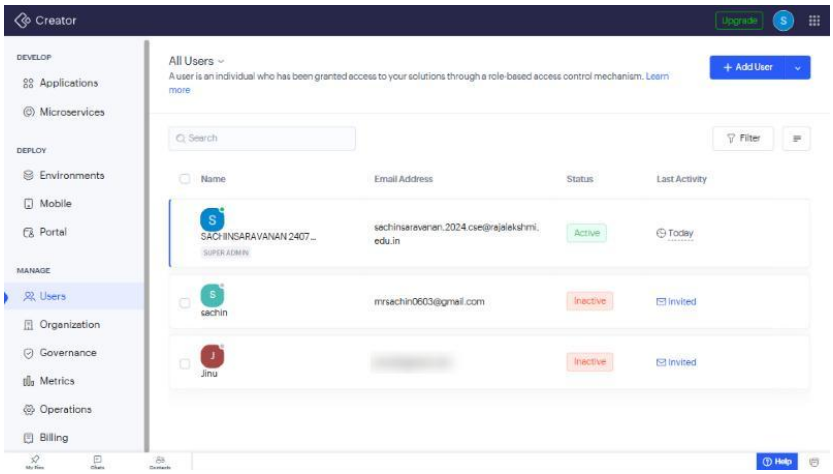


Figure 5.25: Users of the Zoho Creator Account

5.9 AI and Analytics Features

AI. By design, LabLend was built without Zia or the AI application builder; every form, field and script was created manually so that the developer understood and could explain each part. No AI-generated content is used in the application. Possible uses of Zia, such as predicting demand for popular items, are discussed in Section 9.2.

Analytics. Analytics in LabLend are operational rather than predictive. The Dashboard shows three live KPIs (available, issued and overdue items), the Overdue Items report is a permanently filtered view, and the All Requests report can be

grouped, sorted and searched by status, equipment or student to answer questions such as which items are borrowed most often.

5.10 Pro-Code Extension

No pro-code extension (Catalyst function, JavaScript widget or Creator REST API) was used, because every requirement in the brief could be met with configuration and short Deluge scripts. The natural next step that would need pro-code is a QR-code scanning page or widget that lets staff issue and return items by scanning a label on the equipment, or a Catalyst function that synchronises LabLend with a central college inventory database.

5.11 Mobile and Portal Access

The Zoho Creator mobile app gives the same forms and reports on Android and iOS without extra development. The builder's phone preview (Figure 5.26) shows the Equipment form rendered for a phone screen. A customer portal was not configured, because students are expected to be invited users of the college account in a future version rather than external portal users.

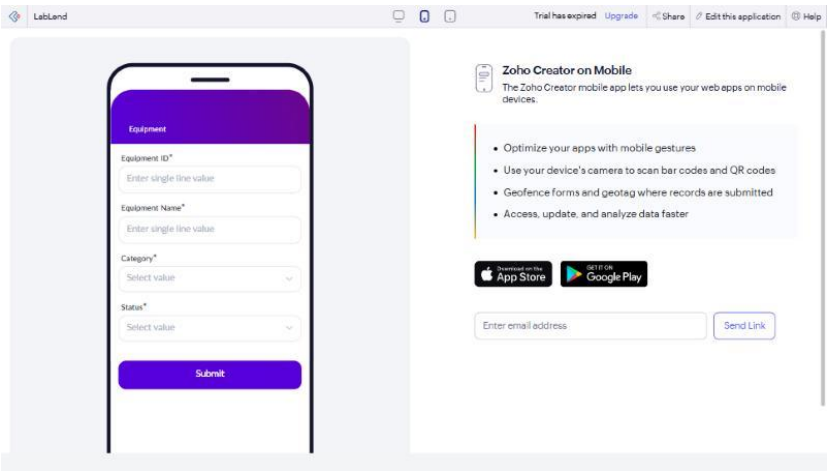


Figure 5.26: Phone Preview of the Application

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done at three levels. Unit testing exercised each form with valid and invalid input and each Deluge script by triggering its event (opening the form, submitting it, pressing the report button). Integration testing followed complete requests through the lifecycle and checked that the equipment status, the overdue flag, the Overdue Items report and the dashboard counts changed together. User acceptance testing ran the reviewer demo script from the project brief: a student submits a request, staff approves and issues it, the dashboard count changes, the item is returned and an overdue example is shown. All test cases in Table 6.1 were executed on 09 November 2026 in the live application.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Add equipment master records	Enter EQ-001 to EQ-005 with category; submit	Five records, status Available	Five items saved as Available	Pass
TC-02	FR-02	Add students	Enter three students with roll no., dept., e-mail	Three records saved	Students saved	Pass
TC-03	FR-04	Defaults on opening request form	Open Requests form	Today's date, Pending, overdue off; locked	Defaults shown and locked	Pass
TC-04	FR-05	Only available items offered	Open Equipment drop-down with one item issued	Issued item not listed	Raspberry Pi 4 not listed	Pass
TC-05	FR-06	Return date before required date	Required 10-Nov, return 08-Nov; submit	Alert; record not saved	Alert shown; submit cancelled	Pass
TC-06	FR-03	Submit valid request	Arun Kumar, Digital Oscilloscope, 10-Nov to 17-Nov	Request 1001, Pending	Request 1001 created as Pending	Pass
TC-07	FR-07	Approve a request	All Requests → 1001 → Approve	Status Approved; Issue button appears	Status Approved	Pass
TC-08	FR-08	Issue a request	1001 → Issue	Request Issued; equipment Issued	Both statuses Issued	Pass
TC-09	FR-10	Issue a request already past due	1002 (return 05-Nov) → Approve → Issue	Overdue = true; listed in Overdue Items	Overdue true; listed	Pass
TC-10	FR-12	Dashboard after two issues	Open Dashboard	3 available / 2 issued / 1 overdue	3 / 2 / 1 shown	Pass
TC-11	FR-09	Return in good condition	Returns → 1001, Good, note; submit	1001 Returned; oscilloscope Available	As expected; dashboard 4 / 1 / 1	Pass
TC-12	FR-09	Only issued requests can be returned	Open Request drop-down in Returns	Only 1002 listed	Only 1002 listed	Pass
TC-13	FR-13	Actions shown by status	Open record menu of a Returned request	No Approve / Reject / Issue	Actions hidden	Pass
TC-14	FR-14	Mobile layout	Open phone preview	Form usable on phone width	Rendered correctly	Pass

6.3 Debugging Deluge Scripts

Three techniques were used. First, every script was saved and then exercised immediately from the live application, and the record was reopened to confirm which fields had changed. Second, the Deluge editor's syntax check on save caught mistakes before they reached users; for example, a condition first written against the field `Condition` failed because the platform had given the drop-down the link name `Condition_field` (Condition is a reserved word), and the script was corrected to use the link name shown in the field properties. Third, defensive status checks were added to every function, so that a second click or a click on a record in the wrong state leaves the data unchanged instead of producing an inconsistent status.

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Process Return script failed to save	Drop-down link name is Condition_field, not Condition	Used the link name from field properties	Closed
BUG-02	Issued item still offered in a request opened earlier	Lookup filter is applied when the form loads	Availability re-checked in Validate Request Dates	Closed
BUG-03	Daily Overdue Check could not be enabled	Schedules need a paid plan	Overdue derived live in report filter and dashboard criteria	Closed (workaround)
BUG-04	Returns drop-down shows only the request number	Lookup display field is Request ID only	Accepted for now; add student and item to display (Sec. 9.2)	Open (minor)

6.5 User Acceptance Testing

The demo script of the project brief was run end to end in the live application, playing the student and the lab staff in turn. Feedback was collected while reviewing the application and is summarised in Table 6.3.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Developer (as student)	Student	Request form is short; locked date and status avoid confusion	Kept as designed
Developer (as lab staff)	Lab Staff	One-click Approve / Issue is faster than editing status	Kept; actions limited by status
Reviewer	Reviewer	Dashboard counts and overdue list on one page are easy to present	Overdue report embedded on dashboard
Reviewer	Lab Staff	Return drop-down should show student and item	Logged as BUG-04 / future enhancement

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

LabLend was developed and tested in the Zoho Creator development environment, which on the free plan is also the environment that users access: changes saved in the builder are live immediately at the application URL. Zoho Creator offers a multi-stage Environments feature (Development → Stage → Production) that lets a developer test changes in Stage and publish them to Production without affecting users. As Figure 7.1 shows, no application has been added to Stage or Production in this account, because the trial that enables environments has expired. The intended strategy for a real deployment is: build and test in Development, publish to Stage for the lab staff to verify with sample data, and then publish version 1.0 to Production for students; later changes follow the same path.

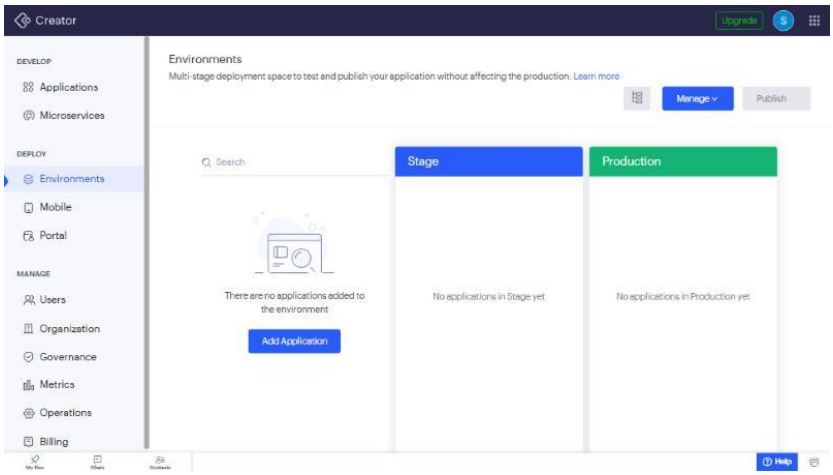


Figure 7.1: Environments (Stage and Production) in the Admin Console

7.2 Performance Optimisation

Performance work focused on keeping each script to the minimum number of fetches, because the volume of data in a single laboratory is modest. Table 7.1 lists the techniques applied. The effect was not measured with timings, so the effect column is qualitative.

Table 7.1: Performance Optimisation Applied

Optimisation Applied	Component	Approach Used	Effect
Filter in the lookup instead of in a script	Requests.Equipment, Returns.Request	Lookup criteria Status == Available / Issued	Only valid choices loaded
Single-record fetch by ID	DS-02, DS-04, DS-05 to DS-07	Fetch with [ID == value]	One indexed read per script

Optimisation Applied	Component	Approach Used	Effect
Derive overdue status at read time	Overdue Items report, Dashboard	Date comparison in criteria	No nightly batch job needed
Return early on wrong status	DS-05 to DS-07	if(Status == …) guard	No unnecessary updates
Criteria fetch in the schedule	DS-08	Requests[Status == "Issued"]	Only issued loans processed

7.3 Security Features

Table 7.2: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Zoho account authentication	Whole application	Anonymous access to lab data
Invitation-based user access	Users area of the admin console	Unknown users viewing or editing records
Read-only system fields	Request Date, Status, Overdue (disabled on load)	Students changing status or dates
Server-side validation	Validate Request Dates (on validate)	Invalid dates; double booking of an item
Status guards in functions	approveRequest, rejectRequest, issueRequest	Out-of-order or repeated actions
Conditional report actions	Approve / Reject / Issue buttons	Actions run on records in the wrong state
Hosted data with platform backups	Zoho India data centre	Loss of the paper register

7.4 Governance and Administration

Naming follows one pattern across the application: forms use singular or plural business names (Equipment, Students, Requests, Returns), link names use underscores (Expected_Return_Date), workflows are named after what they do (Request Form Defaults, Process Return) and functions use verb–noun camel case (issueRequest). Each record keeps the platform's added-time and modified-time metadata, and every request carries its own Request ID, dates and status history in the form of the request and return records, which gives the lab a traceable record of each loan. Data can be exported from any report, and the platform backs up the hosted data. User invitations, the application settings and the scripts are maintained by the application owner. Known gaps to be maintained are listed in Section 8.4.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario below follows the reviewer demo script with the seed data: five equipment items (Digital Multimeter, Digital Oscilloscope, Arduino Uno Kit, Raspberry Pi 4, Digital Microscope) and three students (Arun Kumar – CSE, Priya Sharma – ECE, Rahul Menon – IT). At the start the dashboard showed 5 available, 0 issued and 0 overdue items.

Step 1. Arun Kumar submits request 1001 for the Digital Oscilloscope for signal analysis, required on 10-Nov-2026 and returning on 17-Nov-2026; it is saved as Pending. **Step 2.** Priya Sharma submits request 1002 for the Raspberry Pi 4 for an IoT lab assignment, with an expected return date of 05-Nov-2026 to create an overdue example. **Step 3.** Lab staff approves and issues both requests from the All Requests report; both items become Issued and request 1002 is flagged overdue. The dashboard changes to 3 / 2 / 1. **Step 4.** The oscilloscope is returned in good condition through the Returns form; request 1001 becomes Returned and the oscilloscope Available. **Step 5.** The dashboard settles at 4 / 1 / 1, with request 1002 shown in the overdue list.

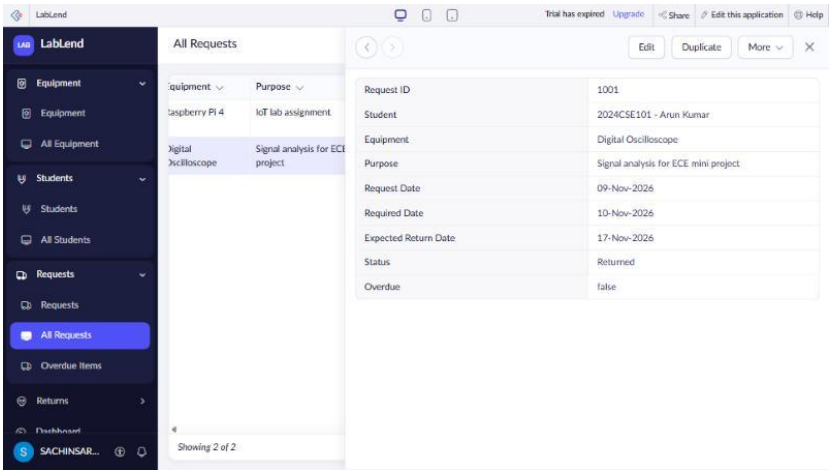


Figure 8.1: Request 1001 after Return (Detail View)

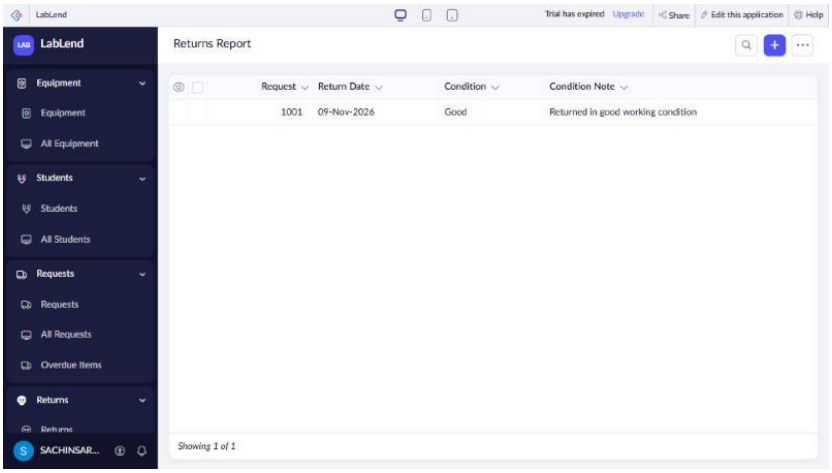


Figure 8.2: Returns Report Showing the Recorded Return

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
1. Data model and four linked forms	Sections 4.2, 4.3, 5.2	Equipment, Students, Requests, Returns with lookups, Auto Number and drop-downs
2. Request lifecycle with Deluge and actions	Sections 5.5, 5.6	Pending → Approved / Rejected → Issued → Returned tested (TC-06 to TC-11)
3. Equipment status kept in sync	Sections 4.4, 5.5	Issue sets Issued; return sets Available / Under Repair; double booking blocked
4. Automatic overdue flag and condition record	Sections 5.3, 5.5	Overdue Items report and flag (TC-09); condition stored on return
5. Reports and live dashboard	Sections 5.3, 5.4	Five reports; dashboard moved 5/0/0 → 3/2/1 → 4/1/1
6. Simple to run and maintain	Sections 5.5, 7.4	Under eighty lines of Deluge in eight named scripts

8.3 Comparison with the Existing System

The comparison is qualitative because no timing study of the manual process was carried out. Availability moves from a physical shelf check to the All Equipment report and the dashboard, which anyone with access can open from a phone. Verbal approval becomes a recorded Pending → Approved step with a single button. The return date, which used to be agreed in conversation, is now a mandatory field validated against the required date. Late loans, which used to be found during stock checks, appear in the Overdue Items report and in red on the dashboard on the day they become late. Return condition, previously unrecorded, is a drop-down that also decides whether the item goes back on the shelf or to repair. Finally, double booking — possible when the logbook lagged behind the shelf — is prevented by the availability filter and the validation script.

Table 8.2: Existing System versus LabLend

Aspect	Existing (Logbook)	LabLend
Checking availability	Walk to lab; check shelf	Open All Equipment or Dashboard
Approval record	Verbal	Status change by Approve / Reject action
Return date	Agreed verbally	Mandatory, validated date
Overdue detection	During stock check	Live report and dashboard count
Return condition	Not recorded	Good / Minor Damage / Needs Repair
Double booking	Possible	Blocked by lookup filter and validation

8.4 Limitations of the Low-Code Approach

Several constraints appeared during the project. Scheduled workflows require a paid plan, so the daily refresh of the stored overdue flag could not be enabled; the design works around this by computing overdue status live, but the stored flag on an issued request is accurate only as of its last update. The free plan also limits the account to one application and disables Stage and Production environments, so changes go straight to users. Page styling is limited to the components of the page builder, and precise criteria on the dashboard counts required editing the page's ZML by hand. Lookup filters are evaluated when the form opens, which is why availability had to be checked again on submission. Permission profiles for separate Student and Lab Staff roles were not configured in this build. Finally, the application depends on Zoho Creator, so moving it to another platform would require rebuilding the forms and Deluge logic.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of LabLend is no-code configuration: forms, fields, mandatory flags, drop-downs, lookups with filters, list reports, the overdue filter, report actions with display conditions, the navigation and the page layout. Low-code Deluge was needed for the parts that change more than one record or must be checked on the server: form defaults, submission validation, return processing and the three status functions. Pro-code would be needed for the next level of capability, such as a camera-based QR scanner for issue and return, or a synchronisation service with a central college inventory database. The point to move to pro-code is when a requirement depends on device access or external processing that the platform components cannot express.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

LabLend is a Zoho Creator application that replaces the paper logbook of a college laboratory with a structured borrow-and-return workflow. It combines four linked forms, five list reports including a live Overdue Items report, a dashboard of available, issued and overdue items, four form workflows, three status-guarded Deluge functions behind one-click report actions, and a saved daily overdue schedule.

The project shows that a complete, traceable lending workflow can be built on a low-code platform by one developer in a short time, with Deluge used only where configuration was not enough. The end-to-end test followed the reviewer demo script exactly — request, approve, issue, dashboard change, return and overdue example — and every step behaved as designed.

9.2 Future Enhancements

- Create Student and Lab Staff permission profiles so that students can only submit requests and view their own records, and only staff can run Approve, Reject, Issue and Returns.
- Enable the Daily Overdue Check schedule on a paid plan and add a reminder e-mail to the student using the address in the Students form.
- Show the student and equipment name in the Returns drop-down, and add a calendar report of loans by required and expected return date.
- Add QR-code labels on equipment and a scanning page in the mobile app for faster issue and return.
- Publish through Stage and Production environments and add usage analytics, such as the most borrowed items per month, using Zoho Analytics or Zia.

9.3 Learning Outcomes and Career Relevance

Building LabLend gave practical experience in data modelling with lookups and filters, designing a status-driven workflow, writing Deluge for form events and functions, connecting functions to report actions, building a dashboard in the page builder and ZML, and working within the limits of a free plan. These skills relate directly to roles such as low-code developer, solutions consultant and business-

process automation engineer, and the habit of enforcing a process both in the interface and on the server applies to any business application.

REFERENCES

[1] R. Nandhini, S. Priya and K. Lakshmi, “Laboratory inventory management system for academic institutions,” *International Journal of Engineering Research & Technology*, vol. 8, no. 5, pp. 512–515, 2019.

[2] J. Kumar and P. Sharma, “Web-based equipment lending and tracking system for engineering laboratories,” *International Journal of Computer Applications*, vol. 176, no. 38, pp. 21–25, 2020.

[3] International Organization for Standardization, “Asset management – Overview, principles and terminology,” ISO 55000:2014, 2014.

[4] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[5] A. C. Bock and U. Frank, “Low-Code Platform,” *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[6] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in *Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 171–178.

[7] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ .

[8] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ .

[9] Zoho Corporation, “Pages and ZML (Zoho Markup Language) – Zoho Creator Help.” [Online]. Available: https://www.zoho.com/creator/help/pages/ .

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	One section per form plus Dashboard	5.1	Figure 5.2
II	Forms with ≥ 8 distinct field types	Y	Single Line, Multi Line, Email, Drop Down, Date, Auto Number, Lookup, Decision Box	5.2	Figure 5.7
II	Lookup field (relationship between forms)	Y	Student, Equipment, Request lookups	4.2, 5.2	Figure 4.2
II	Subform (parent–child data)	N	Not used; child records are separate forms	4.2	—
II	Field validations / layout rules	Y	Mandatory fields, lookup filters, on-validate script	5.2, 5.5	Figure 5.18
II	List report with filter, grouping, custom action	Y	Overdue Items (filter); All Requests (actions)	5.3	Figure 5.14
II	Kanban / Calendar / Timeline / Map report	N	Not used; calendar listed as future work	9.2	—
II	Pivot table / chart report	N	Not used; dashboard counts used instead	5.4	—
II	Page / dashboard built with UI Builder	Y	Dashboard page with KPI panel and embedded report	5.4	Figure 5.15
II	Deluge – form events (load / validate / success)	Y	All three events used	5.5	Code 5.2
II	Deluge – functions, fetch, loops, collections	Y	Three functions; fetch by ID; for each in schedule	5.5	Code 5.3
II	Email / in-app notifications	Partial	alert messages on validation; no e-mail (out of scope)	5.5	Figure 5.18
II	Form workflow	Y	Four form workflows	5.6	Figure 5.21
II	Scheduled workflow	Partial	Daily Overdue Check saved; disabled on free plan	5.6	Figure 5.24
II	Approval process	Partial	Approve / Reject actions instead of approval process	5.6	Figure 5.23
II	Blueprint	N	Not needed for a single-path lifecycle	5.6	—
II	Batch / report workflow	Y	Three report workflows (custom actions)	5.6	Figure 5.22
II	Testing & debugging evidence	Y	14 test cases, defect log	6	Table 6.1
III	Integration with a Zoho service	N	Out of scope	5.7	—
III	Connection / REST API call (invokeurl)	N	Not used	5.7	—
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Partial	Users invited; profiles as future work	5.8	Figure 5.25
III	Record-level access criteria / customer portal	N	Not configured	5.11	—
III	Zia / AI feature	N	Deliberately not used	5.9	—
III	Analytics (charts / Zoho Analytics)	Partial	KPI counts on dashboard	5.4, 5.9	Figure 5.15
IV	Performance optimisation applied & measured	Partial	Applied; not measured with timings	7.2	Table 7.1
IV	Deployment across environments	N	Development only; strategy described	7.1	Figure 7.1
IV	Security controls (encryption, OAuth, permissions)	Partial	Authentication, invitations, read-only fields, guards	7.3	Table 7.2

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Governance (audit trail, administration)	Partial	Naming, record metadata, exports	7.4	—
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used; justified in 5.10	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	—
—	Mobile app / portal access	Y	Zoho Creator mobile app (phone preview)	5.11	Figure 5.26

Total features fully implemented: 13 / 32 (8 partially implemented).

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – DELUGE SOURCE CODE

C.1 Request_Form_Defaults

```
// Script ID : DS-01
// Location  : Requests form – Created – Load of the form
// Purpose   : Default and lock system fields
input.Request_Date = zoho.currentdate;
input.Status = "Pending";
input.Overdue = false;
disable Request_Date;
disable Status;
disable Overdue;
```

C.2 Validate_Request_Dates

```
// Script ID : DS-02
// Location  : Requests form – Created – Validations on form submission
// Purpose   : Block invalid dates and unavailable equipment
if(input.Expected_Return_Date < input.Required_Date)
{
    alert "Expected return date cannot be before the required date.";
    cancel submit;
}
eq = Equipment[ID == input.Equipment];
if(eq.Status != "Available")
{
    alert "Sorry, this equipment is not available right now.";
    cancel submit;
}
```

C.3 Return_Form_Defaults

```
// Script ID : DS-03
// Location  : Returns form – Created – Load of the form
// Purpose   : Default the return date
input.Return_Date = zoho.currentdate;
```

C.4 Process_Return

```
// Script ID : DS-04
// Location  : Returns form – Created – Successful form submission
// Purpose   : Close the request and release the equipment
req = Requests[ID == input.Request];
req.Status = "Returned";
req.Overdue = input.Return_Date > req.Expected_Return_Date;
eq = Equipment[ID == req.Equipment];
if(input.Condition_field == "Needs Repair")
{
    eq.Status = "Under Repair";
}
else
{
    eq.Status = "Available";
}
```

C.5 approveRequest

```
// Script ID : DS-05
// Location  : Function – called by report action Approve Request
void approveRequest(int reqId)
```

```
{
    r = Requests[ID == reqId];
    if(r.Status == "Pending")
    {
        r.Status = "Approved";
    }
}
```

C.6 rejectRequest

```
// Script ID : DS-06
// Location  : Function - called by report action Reject Request
void rejectRequest(int reqId)
{
    r = Requests[ID == reqId];
    if(r.Status == "Pending" || r.Status == "Approved")
    {
        r.Status = "Rejected";
    }
}
```

C.7 issueRequest

```
// Script ID : DS-07
// Location  : Function - called by report action Issue Equipment
void issueRequest(int reqId)
{
    r = Requests[ID == reqId];
    if(r.Status == "Approved")
    {
        eq = Equipment[ID == r.Equipment];
        if(eq.Status == "Available")
        {
            r.Status = "Issued";
            r.Overdue = r.Expected_Return_Date < zoho.currentdate;
            eq.Status = "Issued";
        }
    }
}
```

C.8 Daily_Overdue_Check

```
// Script ID : DS-08
// Location  : Schedule - daily at 00:05, Asia/Kolkata (disabled on free
plan)
// Purpose   : Refresh the stored overdue flag of issued requests
for each  r in Requests[Status == "Issued"]
{
    r.Overdue = r.Expected_Return_Date < zoho.currentdate;
}
```

APPENDIX D – USER MANUAL

Open the LabLend application URL in a browser and sign in with the Zoho account that was invited to the application. On a phone, install the Zoho Creator mobile app, sign in and open LabLend. The left menu has one section per form and a Dashboard.

URL : https://creatorapp.zoho.in/sachinsaravanan.2024.cse1252/lablend/

Lab Staff – setting up. Open Equipment → Equipment and add each item with a unique Equipment ID (for example EQ-006), its name and category; leave the status as Available. Open Students → Students and add each student with roll number, name, department and e-mail. Use All Equipment and All Students to check or edit the lists.

Student – requesting equipment. Open Requests → Requests. The request date and the Pending status are filled in automatically. Choose your name in Student, choose an item in Equipment (only available items are listed), describe the purpose and enter the required date and the expected return date, then click Submit. If the return date is before the required date, or someone else has just taken the item, a message explains why the request was not saved.

Lab Staff – approving and issuing. Open Requests → All Requests. For a Pending request, open the record menu (…) and choose Approve or Reject. When the student collects the item, open the menu of the Approved request and choose Issue; the item is marked Issued and the request moves to Issued.

Lab Staff – recording a return. Open Returns → Returns, choose the request number (only issued requests are listed), confirm the return date, choose the condition (Good, Minor Damage or Needs Repair), add a note and click Submit. The request becomes Returned and the item goes back to Available, or to Under Repair if it needs repair. After a repair, edit the item in All Equipment and set its status to Available.

Everyone – monitoring. Open Dashboard to see the number of available, issued and overdue items and the list of overdue loans. Open Requests → Overdue Items for the full overdue list, and use search and sorting in any report to find a student or an item.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	SACHIN SARAVANAN	Register No.	240701449
Project Title	LabLend: Lab Equipment Borrowing and Return Tracking System	Date	11/11/2026

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.

Level	% of Max. Marks	Descriptor
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	SACHIN SARAVANAN	Register No.	240701449
Department / Section	Computer Science and Engineering	Date of Viva	11/11/2026
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____